AI Influencer Playground v5

Manuale d'uso

Documento interno

October 14, 2026

Abstract

Questo manuale descrive come usare il playground: chat con personaggi sintetici, generazione immagini (SDXL, Qwen-Image, Z-Image, FLUX.2), dataset, pipeline LoRA completa e strumenti di diagnostica. Il sistema è uno strumento interno: i pagamenti PPV sono simulati e non viene pubblicato nulla su piattaforme esterne.

Indice

1 Introduzione **2**

2 Architettura in breve **2**

3 Avvio, accesso e spegnimento **2**

4 La scheda Home **2**

5 La scheda Chat **2**

6 La scheda Immagini **3**

7 La scheda Dataset **3**

8 La scheda LoRA (tre passi) **3**

8.1 1 – Personaggio . . . 3

8.2 2 – Dataset LoRA (reference canoniche e due rami) . . . 3

8.3 3 – Training . . . 4

9 Flusso completo consigliato **4**

10 Modelli e compatibilità **4**

11 Problemi comuni **4**

12 Manutenzione, backup e file importanti **4**

13 Appendice: variabili `.env` principali **4**

1 Introduzione

Il playground serve a costruire e testare influencer sintetiche:
- chat realistiche con fan simulati (memoria per fan, ritardi, orari, *Human mode*);
- generazione e curatela di immagini NSFW con ComfyUI;
- creazione di un dataset coerente per personaggio;
- training di una LoRA di identità con kohya (worker sul server) e suo riuso in generazione.

Vincoli: solo personaggi adulti e completamente sintetici, nessuna likeness di persone reali; pagamenti simulati (nessun circuito reale); Fanvue/Instagram non sono collegati.

2 Architettura in breve

- **WebUI** (React) servita dall'API su `http://192.168.1.60:8010`.
- **API** (FastAPI) + **PostgreSQL** in Docker (progetto `ai-influencer`).
- **Ollama** per chat/embedding, **ComfyUI** per le immagini (entrambi già installati sul server).
- **Trainer LoRA** host-side (`scripts/trainer/train_worker.py`) che usa kohya sd-scripts su GPU con memoria libera.
- Dati e media su disco in `data/` (media, dataset, pose, LoRA).

3 Avvio, accesso e spegnimento

- Avvio: `cd backend-v6 && ./start.sh` (costruisce la WebUI, applica le migrazioni, avvia API e worker).
- Stato: `docker compose ps`; log: `docker compose logs -f api` (o `worker`).
- Accesso: aprire l'URL nel browser e inserire la `API_KEY` del file `.env` (resta nella sessione del browser).
- Arresto senza perdere dati: `docker compose stop`; riavvio: `docker compose up -d`.
- Trainer: `scripts/trainer/train_worker.py` (oppure servizio systemd già predisposto).

4 La scheda Home

La pagina iniziale mostra lo stato del sistema:
- **Flag colorati** per API+database, ComfyUI, Ollama, telemetria GPU e worker di training (heartbeat): un flag rosso/giallo indica dove intervenire.
- **GPU in tempo reale** (memoria, utilizzo, potenza, temperatura, processi) con pulsante **Libera VRAM** (scarica i modelli Ollama e la cache di tutte le istanze ComfyUI). La generazione in batch usa **4 istanze ComfyUI** (una per GPU): i tempi si dividono quasi per quattro.
- **Personaggi** con foto profilo, stile e numero di LoRA (link diretto alla chat).
- **Modelli disponibili**: checkpoint ComfyUI (famiglia, compatibilità), LoRA e modelli Ollama.

5 La scheda Chat

- Creare/gestire **fan** e **conversazioni**; il personaggio può scrivere per primo.
- **Realismo**: ritardo casuale per personaggio e orari di attività; le risposte in coda sopravvivono ai riavvii.
- **Human mode**: risposte multiple, refusi con correzione, messaggi spontanei di ripresa.
- **PPV simulato**: foto bloccate con prezzo; il fan paga (simulato) e sblocca.
- **Foto manuale**: invio di una foto generata (con scena e didascalia), anche bloccata.
- **Annulla generazione** disponibile in ogni punto in cui si genera un'immagine; eliminazione conversazione dalla lista.

6 La scheda Immagini

- **Checkpoint**: SDXL real/pony/anime, Qwen-Image, Z-Image, FLUX.2 (i modelli video sono nascosti). Il selettore mostra famiglia e compatibilità.
- **LoRA multiple**: ogni LoRA abilitata ha **categoria** (Personaggio, Stile, Outfit, Composizione, Altro), **peso modello** e **CLIP weight** (vuoto = come il modello). Le LoRA incompatibili con la famiglia del checkpoint vengono nascoste.
- **Pose Library**: scegli una o più pose; la generazione cicla sulle pose selezionate (forza regolabile).
- **Preset**: salva/applica/elimina combinazioni complete (checkpoint, LoRA, pose, prompt, conteggio).
- **Libreria**: approva/rifiuta, voto a stelle, didascalia e tag, visore a schermo intero, riuso nelle chat.
- La **LoRA attiva del personaggio** viene abilitata automaticamente quando la famiglia del checkpoint corrisponde.

7 La scheda Dataset

- **Fonti**: canale Telegram pubblico, elenco di URL, cartella in `data/imports`, **profilo Instagram pubblico** (via Scrapling: `@profilo` o URL). I video vengono scaricati con anteprima e marcati come video. Le immagini da Instagram servono solo come riferimento di posa/stile (moodboard), mai come volto da replicare: l'uso è a tuo carico nel rispetto dei ToS di Meta. Se Instagram chiede il login, imposta `INSTAGRAM_COOKIES` nel `.env`.
- **Classifica non classificate**: descrive con AI le immagini mancanti e ritenta le fallite; se un run si interrompe, il pulsante riparte.
- **Dettagli immagine**: descrizione, campi strutturati (posa, scena, luce...), tag, **Estrai posa** (salva lo skeleton nella Pose Library).
- **Esporta JSONL** e **Genera profilo** (bozza di personaggio dalle descrizioni).

8 La scheda LoRA (tre passi)

8.1 1 – Personaggio

- Scegli il **modello** per la foto profilo e usa *Genera/Rigenera da zero*, oppure *Modifica con Qwen-Edit* (prompt tipo `same person, studio portrait`).
- Azioni: **Duplica**, **Elimina** (pulisce anche LoRA, dataset e job), + nella barra sinistra per creare un nuovo personaggio.
- A destra puoi aprire la **guida** (Mostra/Nascondi): spiega parametri, checkpoint, pesi e glossario passo per passo.

8.2 2 – Dataset LoRA (reference canoniche e due rami)

- **1 · Identità**: genera 20–50 candidati con **Qwen-Image-2512** (selezionabile). Il campo prompt è precompilato; le scene del ramo variazioni sono 20 di esempio, ripristinabili con *Ripristina default*. La sezione richiudibile **Prompt del personaggio** mostra la parte identità del prompt della foto profilo (soggetto e aspetto), la salva e la include nei candidati, così etnia e tratti non si perdono. Nel passo dei candidati è selezionabile anche **Playground v2.5** (SDXL): riferimenti e pose funzionano come nel ramo SDXL. Nella scheda LoRA è disponibile la famiglia **qwen-image**: il training usa musubi-tuner con pre-cache e block swap, a partire dai pesi bf16 del modello (i checkpoint fp8 vengono mappati automaticamente).

- **2 · Reference canoniche**: assegna un **ruolo** a 4–8 immagini (volto ravvicinato, 3/4, profilo, mezzo busto, figura intera frontale, 3/4, laterale). Tutte le generazioni successive usano queste reference, non la sola foto profilo.
- **3 · Due rami indipendenti**: **Variazioni Qwen-Image-Edit** (una scena per riga, modello selezionabile con LoRA Lightning abbinata) e **Candidati SDXL** (checkpoint + IPAdapter + pose ControlNet). I due rami non si incrociano e confluiscono nella stessa griglia.
- **Curatela**: seleziona le immagini, correggi le didascalie, elimina; la barra mostra `selezionate/60` (obiettivo 50–70 immagini finali).
- **Avvisi qualità**: duplicati, poche reference, poche inquadrature, stessa posa/scena ripetuta.

8.3 3 – Training

- Crea la voce LoRA (nome, famiglia, trigger, checkpoint base, **rank**: capacità della LoRA, tipico 16–32 per i personaggi).
- Accoda il training scegliendo il **dataset curato** (minimo 4 selezionate, consigliate 20+).
- Parametri base/avanzati: risoluzione, passi, learning rate, ripetizioni, batch, prefisso caption, salvataggi, GPU.
- Segui l'avanzamento (passo, loss, log) e annulla se serve; a fine training la LoRA è copiata in ComfyUI e in `data/loras`.
- Attiva la LoRA (la prima pronta di una famiglia si attiva da sola).

9 Flusso completo consigliato

1. Crea il personaggio (scheda LoRA, +) e genera la **foto profilo** (scegli il modello; eventualmente modificala con Qwen-Edit).
2. Crea un **dataset** e genera 30–100 **variazioni** con Qwen-Image-Edit (scene diverse, stessa persona).
3. Se vuoi, aggiungi **candidati SDXL** per allineare il look alla famiglia della LoRA.
4. **Seleziona** le immagini più coerenti (obiettivo 20+, meglio 40–80) e correggi le didascalie.
5. Controlla gli **avvisi qualità** e i duplicati.
6. Crea la LoRA e **accoda il training** sul dataset curato.
7. Attiva la LoRA e usala nella scheda **Immagini**: scegli checkpoint della stessa famiglia, eventualmente aggiungi stile/outfit e pose.
8. Curati i risultati nella libreria e riusali in chat.

10 Modelli e compatibilità

11 Problemi comuni

12 Manutenzione, backup e file importanti

- **Backup**: snapshot in `backups/<versione>/` (dump PostgreSQL, `.env`, sorgente). Prima di modifiche importanti crearne uno nuovo.
- **Aggiornamenti**: modificare il codice in `backend-v6`, poi `docker compose up -d -build`.
- **Percorsi utili**: `app/` (API), `webui/src` (interfaccia), `workflows/` (grafi SDXL), `scripts/` (download modelli, trainer), `data/` (media, dataset, pose, LoRA).
- **Log**: `docker compose logs`; trainer in `/data/data_ssd/pigni/trainer/runs/<job>/train.log`.

13 Appendice: variabili `.env` principali

Famiglia	Modelli	Note
SDXL real	PornMaster Pro, LUSTIFY	LoRA personaggio, IPAdapter, ControlNet posa, FaceDetailer
SDXL real (SFW)	Playground v2.5	realistico-estetico per foto quotidiane non esplicite; LoRA SDXL compatibili
SDXL pony	(non installati al momento)	score tag, workflow pony
SDXL anime	(non installati al momento)	tag danbooru, workflow anime
Qwen-Image	qwen_image fp8, Qwen-Image-Edit 2509	text2img 20 step; Edit per variazioni/avatar
Z-Image	z_image bf16, turbo bf16	turbo 4 step (veloce), base 30 step
FLUX.2	flux2 dev fp8 + turbo LoRA	8 step, ottima qualità, primo caricamento lento

Table 1: Famiglie supportate e uso. IPAdapter/ControlNet/LoRA personaggio sono solo SDXL.

Sintomo	Rimedio
ComfyUI non raggiungibile	Avvia ComfyUI sul server; il flag in Home diventa rosso.
Ollama non raggiungibile	Avvia Ollama; verifica con il flag in Home.
Classificazione dataset ferma	Premi *Classifica non classificate*: riparte anche se bloccata.
Training in coda ma non parte	Controlla il flag *Trainer LoRA*: avvia `train_worker.py`.
Generazione bloccata/lenta	Usa *Annulla* nella UI; *Libera VRAM* in Home.
Spazio disco	I modelli sono in `/data/data_ssd/pigni/comfyUI`; i media in `data/`.
Instagram chiede il login	Imposta `INSTAGRAM_COOKIES` nel `.env` oppure usa URL/cartella.

Variabile	Significato
`API_KEY`	Chiave di accesso alla WebUI/API.
`POSTGRES_PASSWORD`	Password del database (volume persistente).
`DATABASE_URL`	Connessione al database usata dai container.
`TEXT_PROVIDER` / `OLLAMA_URL`	Provider chat e indirizzo Ollama.
`IMAGE_PROVIDER` / `COMFYUI_URL`	Provider immagini e indirizzo ComfyUI.
`CHAT_IMAGE_ENABLED`, `CHAT_IMAGE_NSFW`	Foto in chat e contenuti NSFW.
`PPV_ENABLED`	Pagamenti simulati.
`HUMAN_MODE_PROACTIVE_*`	Messaggi spontanei della Human mode.
`LORA_DIR`, `LORA_COMFYUI_SUBDIR`	Archivio LoRA e sotto-cartella in ComfyUI.
`POSE_DIR`	Skeleton della Pose Library.
`INSTAGRAM_COOKIES`	Cookie di sessione Instagram per l'import dei profili (opzionale).
`COMFYUI_URLS`	Istanze ComfyUI aggiuntive (8189–8191) per la generazione parallela.